

MiniOps.jl: Operator and Solver Reference

Mauricio D. Sacchi

October 1, 2026

1 Purpose

MiniOps is a compact matrix-free operator package for signal processing and inverse problems. An operator A stores a forward action and its adjoint, so that the principal syntax is

$$y = Ax, \quad x_a = A^*y,$$

implemented in Julia as $A * x$ and $A' * y$. Operators can also be composed without explicitly forming matrices.

2 Public API

The table below lists the symbols exported by `MiniOps.jl`. This is the list to check when deciding what belongs to the supported public API.

Name	Purpose
<code>Op</code>	Matrix-free linear operator containing forward and adjoint actions, flattened sizes, and a name.
<code>with_shape</code>	Infer flattened operator dimensions from a prototype input.
<code>conv1d_op</code>	Full 1-D convolution with a fixed filter; adjoint is correlation with the conjugated filter.
<code>conv1d_cols_op</code>	Apply the same 1-D convolution independently to every matrix column/trace.
<code>sampling_op</code>	Scalar/vector sampling for 1-D inputs; for arrays of dimension ≥ 2 , sample complete traces while preserving dimension 1 as time.
<code>trace_sampling_op</code>	Explicit whole-trace sampling for arrays shaped $(n_t, n_2, n_3, \dots)$. Recommended when trace semantics should be unambiguous.
<code>blending_op</code>	Delay and sum source traces in a common-receiver gather, with fractional delays handled by linear interpolation.
<code>scaling_op</code>	Multiply an array by a scalar; the adjoint uses the complex conjugate scalar.
<code>diag_op</code>	Elementwise diagonal operator; the adjoint uses conjugated weights.
<code>fft_op</code>	N-D FFT operator, unitary by default; planned and unplanned constructors are available.

Name	Purpose
<code>pad_op</code>	Zero-pad an N-D array; the adjoint extracts the unpadded interior.
<code>nudft_nffts</code>	Number of FFTs required by the Taylor-series NUDFT approximation for a given dimension and order.
<code>nudft1_op</code>	1-D Taylor/FFT approximation to the nonuniform DFT.
<code>nudft2_op</code>	2-D Taylor/FFT approximation to the nonuniform DFT.
<code>nudft3_op</code>	3-D Taylor/FFT approximation to the nonuniform DFT.
<code>nudft_txy_op</code>	Time-space operator combining a spatial 2-D NUDFT with a unitary FFT/IFFT in time.
<code>radon_tx_parab_op</code>	Parabolic Radon transform in (t, x) using linear time interpolation.
<code>radon_tx_tp_chirp_op</code>	Frequency-domain linear Radon transform using chirp factorization.
<code>adjoint_test</code>	Numerical check of $\langle Ax, y \rangle = \langle x, A^*y \rangle$.
<code>linearity_test</code>	Numerical check of operator linearity.
<code>opnorm_power</code>	Power-iteration estimate of $\ A\ _2$.
<code>is_selfadjoint</code>	Numerical comparison of Ax and A^*x .
<code>soft_threshold</code>	Elementwise soft thresholding.
<code>hard_threshold</code>	Elementwise hard thresholding.
<code>ista</code>	Iterative soft-thresholding solver for an ℓ_1 -regularized least-squares problem.
<code>iht</code>	Iterative hard-thresholding solver for sparse inverse problems.
<code>cglsl</code>	Conjugate-gradient least-squares solver with quadratic regularization.
<code>seismic_wavelet</code>	Generate a normalized decaying sinusoidal seismic wavelet with edge tapering.
<code>edge_cosine_taper</code>	Apply a cosine/squared-sine taper to both ends of a vector.

3 Sampling operators

This part of the API has two deliberately different meanings.

3.1 Scalar/vector sampling

For a vector $x \in \mathbb{R}^n$ and indices $i_1, \dots, i_{n_s}$,

$$Sx = \begin{bmatrix} x_{i_1} & \cdots & x_{i_{n_s}} \end{bmatrix}^T.$$

The adjoint scatters the values back to a length- n vector. If an index is repeated, its adjoint contributions are summed.

```
n = 100
idx = [1, 5, 20, 75]
```

```

S = sampling_op(idx, n)

x = randn(n)
d = S * x                # length(idx)
xa = S' * d               # length_n

```

3.2 Whole-trace sampling for 2-D through N-D arrays

For seismic-style arrays, dimension 1 is time. Let

$$\text{size}(X) = (n_t, n_2, n_3, \dots, n_N), \quad n_{\text{tr}} = \prod_{j=2}^N n_j.$$

Internally MiniOps uses

$$X_2 = \text{reshape}(X, n_t, n_{\text{tr}}).$$

Then

$$SX = X_2[:, \text{idx}],$$

so the forward result always has size

$$n_t \times n_s, \quad n_s = \text{length}(\text{idx}).$$

The adjoint returns an array with the original full shape and sums repeated trace indices.

```

# (time, x)
X2 = randn(1000, 120)
S2 = trace_sampling_op([1, 20, 75], size(X2))
D2 = S2 * X2                # 1000 x 3
X2a = S2' * D2               # 1000 x 120

# (time, x, y)
X3 = randn(1000, 30, 20)
S3 = trace_sampling_op([1, 31, 200], size(X3))
D3 = S3 * X3                # 1000 x 3
X3a = S3' * D3              # size(X3)

# (time, x, y, z, component)
X5 = randn(500, 10, 8, 4, 3)
S5 = trace_sampling_op([1, 17, 200], size(X5))
D5 = S5 * X5                # 500 x 3
X5a = S5' * D5              # size(X5)

```

For spatial coordinates such as (ix,iy), convert to the flattened trace number with Julia's column-major convention:

```
j = LinearIndices((nx, ny))[ix, iy]
```

The generic call `sampling_op(idx, size(X))` is backward-compatible and delegates to whole-trace sampling when `ndims(X) >= 2`.

4 Operator examples

4.1 Convolution

```
h = [1.0, -0.5, 0.25]
C = conv1d_op(h)
y = C * x
xa = C' * y
```

For a gather with time along rows and traces along columns:

```
C = conv1d_cols_op(h)
Y = C * X
Xa = C' * Y
```

4.2 Scaling and diagonal weights

```
S = scaling_op(0.5)
y = S * x

D = diag_op(w)
y = D * x
xa = D' * y
```

Both operators use complex conjugation in their adjoints.

4.3 FFT

```
F = fft_op()                # unitary by default
X = F * x
x2 = F' * X
```

With the default normalization,

$$Fx = \frac{1}{\sqrt{N}} \text{fft}(x), \quad F^* X = \frac{1}{\sqrt{N}} \text{bfft}(X).$$

Therefore $F' * (F * x)$ reproduces x to roundoff.

4.4 Padding

```
P = pad_op((100, 60), (20, 10), (20, 10))
Y = P * X
X2 = P' * Y
```

4.5 Blending

```
source_times = [0.0, 0.37, 0.81]
B = blending_op(source_times, dt, nt)
b = B * D                # D is nt x ns
Da = B' * b
```

5 NUDFT normalization

The Taylor NUDFT operators use a *unitary* Fourier convention. In one dimension,

$$Ux = \frac{1}{\sqrt{N}} \text{fft}(x), \quad U^*X = \sqrt{N} \text{ifft}(X).$$

Consequently, when all coordinate perturbations are zero, `nudft1_op(zeros(N), dx)` reduces to the unitary inverse transform U^* , not Julia's raw `ifft`. Thus

$$AX = \sqrt{N} \text{ifft}(X) \quad (\delta x = 0).$$

A direct comparison against `ifft(X)` will therefore differ by $\sqrt{N}$. This is a normalization convention, not an irregular-sampling error. The same convention is used by the multidimensional Taylor NUDFT.

```
N = 128
A = nudft1_op(zeros(N), dx)
X = randn(ComplexF64, N)

x1 = A * X
x2 = sqrt(N) .* ifft(X)
@show norm(x1 - x2) / norm(x2)
```

6 Radon operators

6.1 Parabolic Radon

```
R = radon_tx_parab_op(dt, h, q; href=1000.0)
d = R * m # m: nt x nq
a = R' * d # a: nt x nq
```

6.2 Linear Radon by chirp factorization

```
R = radon_tx_tp_chirp_op(dt, x0, dx, nx, p0, dp, np;
                        f1=2.0, f2=60.0)
d = R * m
ma = R' * d
```

7 Diagnostics

Every newly written operator should first pass a linearity test and then an adjoint test.

```
oklin, errlin = linearity_test(A, x1, x2)
okadj, erradj = adjoint_test(A, x, y)

sigma = opnorm_power(A, x; niter=30)
```

For a correct double-precision implementation, the adjoint error is usually close to machine precision, although interpolation, FFT conventions, and single precision can justify a looser tolerance.

8 Sparse solvers

8.1 ISTA

```
sigma = opnorm_power(A, u0)
step = 0.9 / sigma^2
u = ista(A, y, u0, mu, step; niter=200)
```

This minimizes an ℓ_1 -regularized least-squares objective using soft thresholding.

8.2 IHT

```
u = iht(A, y, u0, tau, step; niter=200)
```

IHT uses hard thresholding and is useful when explicit sparsity is desired.

8.3 CGLS with quadratic regularization

```
x = cgls(A, b, mu, x0; tol=1e-6, max_iter=500)
```

The implemented objective is proportional to

$$\|Ax - b\|_2^2 + \mu\|x\|_2^2.$$

9 Source-file inventory and maintenance notes

- `core.jl`: operator type and shape helper.
- `algebra.jl`: adjoint, composition, scalar multiplication.
- `conv.jl`: 1-D and columnwise convolution.
- `sampling.jl`: scalar and whole-trace sampling.
- `blending.jl`: seismic blending.
- `scaling.jl`: scalar and diagonal operators.
- `fftops.jl`: FFT and padding operators.
- `nudftops.jl`: Taylor-series NUDFT operators.
- `radon_parab.jl`: parabolic Radon operator.
- `radon_chirp.jl`: chirp-factorized linear Radon operator.
- `diagnostics.jl`: operator validation and norm estimation.
- `solvers.jl`: ISTA, IHT, and CGLS.
- `extras.jl`: wavelet and taper utilities.

The file `radon_linear.jl` is currently a legacy duplicate of the parabolic implementation under older names. It is not included by `MiniOps.jl` and therefore is not part of the active package API. It should either remain clearly marked as legacy or be removed after any needed comparison with older notebooks.

10 Recommended validation before release

1. Run `Pkg.test()` under the target Julia version.
2. Run adjoint tests with both real and complex data for operators that support complex arithmetic.
3. Keep the 2-D through 5-D whole-trace sampling tests; they define the intended time-first sampling contract.
4. When comparing NUDFT results with `ifft`, account explicitly for the unitary $\sqrt{N}$ normalization.
5. Keep only active package files in release archives; editor swap files, `_MACOSX`, and repository metadata are not needed.